



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

ASSIGNMENT- 1

Name: Ramneek Kaur

Section: 23AML-2 (B)

UID: 23BAI70043

Subject: Full Stack Development-II

1) Benefits of Using Design Patterns in Frontend Development

Design patterns are standardized and reusable solutions to common software design problems. In frontend development, they help structure applications efficiently and maintain long-term scalability.

Major Benefits:

1. Reusability

Design patterns allow developers to reuse proven solutions instead of writing logic from scratch.

2. Maintainability

Structured code is easier to debug, extend, and refactor.

3. Scalability

Applications built using patterns can grow without becoming unmanageable.

4. Separation of Concerns

Patterns divide responsibilities clearly (UI, logic, data handling).

5. Improved Team Collaboration

When a team follows common design patterns, communication and development speed improve.

6. Reduced Technical Debt

Proper structure avoids messy code accumulation

2) Difference Between Global State and Local State in React

In React applications, state management is crucial for handling dynamic data.

Local State

Local state is confined to a single component and is managed using hooks like `useState`.

Characteristics:

- Component-specific
- Lightweight
- Suitable for UI interactions

Example Use Cases:

- Form inputs
- Toggle buttons
- Modal visibility

Global State

Global state is shared across multiple components and managed using tools like Redux or Context API.

Characteristics:

- Shared across components
- Centralized data storage
- Suitable for large applications

Example Use Cases:

- User authentication
- Theme management
- Project data

Comparison Table

Basis	Local State	Global State
Scope	Single Component	Entire Application
Complexity	Low	Medium/High
Best For	UI logic	Shared data
Management Tool	useState	Redux / Context

3) Comparison of Routing Strategies in Single Page Applications

Routing determines how users navigate through pages in a web application.

1. Client-Side Routing (CSR)

In this approach, routing is handled by JavaScript in the browser.

Advantages:

- Faster navigation
- No full page reload
- Smooth user experience

Disadvantages:

- SEO limitations

- Heavy initial load

Suitable For:

- Dashboards
- Admin panels
- Internal applications

2. Server-Side Rendering (SSR)

In SSR, HTML is generated on the server for each request.

Advantages:

- Better SEO
- Faster initial content rendering

Disadvantages:

- Increased server load
- More complex implementation

Suitable For:

- E-commerce websites
- Blogs
- SEO-focused platforms

3. Hybrid Routing

Hybrid combines CSR and SSR techniques.

Advantages:

- Balanced performance
- SEO-friendly
- Flexible

Disadvantages:

- High architectural complexity

Routing Strategy Diagram

User Request

|

Server

|

CSR → JS Handles Navigation

SSR → Server Generates HTML

Hybrid → Combination of Both

4) Common Component Design Patterns

1. Container–Presentational Pattern

This pattern separates business logic from UI components.

- Container Component → Handles data and logic.
- Presentational Component → Handles UI rendering.

Use Case:

Large scalable applications where logic and UI separation is important.

2. Higher-Order Components (HOC)

A Higher-Order Component is a function that takes a component and returns a new enhanced component.

Use Case:

- Authentication
- Logging
- Permission checks

3. Render Props Pattern

In this pattern, a function is passed as a prop to share logic between components.

Use Case:

Reusable logic such as data fetching or mouse tracking.

Pattern Comparison Table

Pattern	Purpose	Best Use Case
Container	Logic separation	Large apps
HOC	Add extra features	Auth systems
Render Props	Share logic	Reusable utilities

5) Development of Responsive Navigation Bar Using Material UI

A responsive navigation bar adjusts its layout according to screen size.

Key Components Used:

- AppBar
- Toolbar
- Typography
- Button
- Drawer
- IconButton

Responsive Strategy:

Material UI uses breakpoints:

- xs (extra small)
- sm (small)
- md (medium)
- lg (large)
- xl (extra large)

Working Principle:

- On large screens → Display navigation buttons.
- On small screens → Hide buttons and show menu icon.
- Drawer component opens for mobile navigation.

Layout Diagram

Desktop View:

| Logo | Dashboard | Projects | Profile |

Mobile View:

| Logo | ☰ |

|
Drawer Opens

6) Complete Frontend Architecture for Collaborative Project Management Tool

A collaborative project management tool requires scalable and real-time frontend architecture.

a) SPA Structure with Nested Routing and Protected Routes

Suggested Folder Structure:

Copy codesrc/

```
├─ pages/  
├─ components/  
├─ routes/  
├─ store/  
└─ services/
```

Nested Routing:

Allows hierarchical views such as:

- Projects
- Tasks inside projects

Protected Routes:

Only authenticated users can access dashboard pages.

b) Global State Management using Redux Toolkit

Slices:

- Auth Slice
- Project Slice

- Task Slice
- Notification Slice

Middleware:

- Thunk (Async operations)
- Logger (Debugging)
- WebSocket middleware (Real-time updates)

c) Responsive UI Design with Material UI

Features:

- Custom theming
- Dark/Light mode
- Grid-based layout
- Breakpoint handling

d) Performance Optimization Techniques

1. Code Splitting
2. Lazy Loading
3. Memoization
4. Virtualization for large task lists
5. Pagination

e) Scalability and Multi-User Concurrent Access

Real-Time Technology:

- WebSockets
- Firebase Realtime Database

Techniques:

- Optimistic UI updates
- Conflict resolution strategies
- Server-side caching

- Load balancing

Conclusion

Using structured frontend architecture, design patterns, proper state management, and optimized routing strategies ensures:

- High performance
- Scalability
- Maintainability
- Real-time collaboration
- Responsive user experience

A well-designed frontend system reduces complexity and supports future growth effectively.